ANSWER KEY · 2026

Web Programming II Mock Exam Answers

Reference solutions for the 2026 mock paper. Use alongside your notes when revising — the reasoning matters as much as the final answer.

Paper Mock 2026
Questions 23 + Concepts
Issued Apr 2026

Part I · CSS & Layout

Q1 – Q4

Q1 CSS Layout

FILL IN THE BLANKS

1. `grid-template-areas`
2. `align-items`
3. `repeat(auto-fit, minmax(…))` in `grid-template-columns`
4. `flex-grow`

Q2 BEM Naming Convention

MULTIPLE CHOICE

- ✓ `.card__title--active`

Q3 CSS Custom Properties

SELECT ALL THAT APPLY

- ✅ They can be updated dynamically with JavaScript at runtime.
- ✅ They are scoped and can be overridden within specific selectors.
- ✅ They improve maintainability by centralising repeated values.

Q4 CSS Grid + Flexbox

ESSAY

Use **CSS Grid** for the page-level layout — it handles two dimensions (rows + columns):

```css
.page {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 250px 1fr;
}
```

Use **Flexbox** inside card components — it handles one dimension, distributing content:

```css
.card { display: flex; flex-direction: column; justify-content: space-between; }
```

TL;DR Grid = **macro** layout (2D control). Flexbox = **micro** layout (1D flow). They are complementary.

Part II · JavaScript & TypeScript

Q5 – Q8

Q5 Async & the Event Loop

MULTIPLE CHOICE

- ✅ The browser handles async operations via Web APIs, and resolved callbacks are queued and processed by the Event Loop.

Q6 TypeScript

FILL IN THE BLANKS

1. `interface`
2. `string | number`
3. `Partial<T>`
4. assertion (`as Type`)
5. nullable

Q7 Async JavaScript

SELECT ALL THAT APPLY

- ✅ `async/await` is syntactic sugar built on top of Promises.
- ✅ If a Promise is rejected and there is no `.catch()`, it results in an unhandled rejection.
- ✅ The Event Loop processes microtasks (Promises) before macrotasks (`setTimeout`).

Q8 TypeScript Bug Fix

ESSAY

a) Two bugs

1. **`response.json()` is not awaited** — it returns a `Promise`, so `data.name` is `undefined` and `.toUpperCase()` throws.
2. **No error handling** — network errors or non-OK HTTP responses (404 / 500) are unhandled and crash silently.

b) Fixed function

```typescript
interface User {
  id: string;
  name: string;
}

async function loadUserData(userId: string): Promise<string> {
  try {
    const response = await fetch(`/api/users/${userId}`);
    if (!response.ok) throw new Error(`HTTP error: ${response.status}`);
    const data: User = await response.json(); // awaited
    return data.name.toUpperCase();
  } catch (error) {
    console.error("Failed to load user:", error);
    throw error;
  }
}
```

Part III · React Hooks & State

Q9 – Q14

Q9 `useEffect` Dependency Array

MULTIPLE CHOICE

- ✅ `[selectedId]` — runs on mount and whenever `selectedId` changes.

Q10 `useRef` Use Cases

SELECT ALL THAT APPLY

- ✅ Storing a reference to a DOM element to programmatically focus it.
- ✅ Keeping track of a previous state value without causing re-renders.
- ✅ Storing a timer ID from `setInterval` so it can be cleared later.

Q11 Sharing State Across Nesting Levels

MULTIPLE CHOICE

- ✅ Use the Context API with `createContext` and `useContext`.

Q12 React State

FILL IN THE BLANKS

1. re-render
2. prop drilling
3. `useCallback`
4. lift

Q13 Global State Management

SELECT ALL THAT APPLY

- ✅ Zustand has a simpler API than Redux and requires less boilerplate.
- ✅ Redux Toolkit (RTK) is the modern recommended way to use Redux.
- ✅ Zustand stores can be accessed outside of React components.

Q14 Shopping Cart State

ESSAY

a) Comparison

	CONTEXT API	ZUSTAND	REDUX TOOLKIT
Advantage	Zero dependencies, built-in.	Minimal boilerplate, works outside React, easy persistence.	DevTools, middleware, predictable flow.
Disadvantage	Re-renders all consumers on every update.	Less opinionated, no built-in DevTools.	Most boilerplate, steeper learning curve.

b) Implementation — Zustand

```typescript
interface CartItem { id: string; name: string; price: number; quantity: number; }
interface CartStore {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  removeItem: (id: string) => void;
  updateQuantity: (id: string, qty: number) => void;
}

const useCartStore = create<CartStore>()(
  persist(
    (set) => ({
      items: [],
      addItem: (item) => set((s) => {
        const exists = s.items.find((i) => i.id === item.id);
        return exists
          ? { items: s.items.map((i) => i.id === item.id ? { ...i, quantity: i.quantity
          : { items: [...s.items, { ...item, quantity: 1 }] };
      }),
      removeItem: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
      updateQuantity: (id, qty) => set((s) => ({ items: s.items.map((i) => i.id === id
    }),
    { name: "cart-storage" }
  )
);

// Usage in any component:
const { items, addItem } = useCartStore();
```

Part IV · Forms with `react-hook-form`

Q15 - Q17

Q15 `react-hook-form` vs `useState`

MULTIPLE CHOICE

- ✅ It reduces unnecessary re-renders by using uncontrolled inputs and only re-renders on validation events.

Q16 `react-hook-form`

FILL IN THE BLANKS

1. `useForm`
2. `register`
3. `formState.errors`
4. `onChange`
5. `resolver`
6. `handleSubmit`

Q17 Valid `register()` Rules

SELECT ALL THAT APPLY

- ✅ `required`
- ✅ `minLength`
- ✅ `pattern`

Part V · Next.js (Server / Client)

Q18 – Q22

Q18 Next.js Server Action

MULTIPLE CHOICE

- ✅ An async function marked with `"use server"` that runs on the server and can be called directly from a Client Component or form.

Q19 When an API Route is needed

SELECT ALL THAT APPLY

- ✅ Handling a webhook from an external service like Stripe.
- ✅ Exposing an endpoint that a mobile app or third-party service needs to call.
- ✅ Building a public REST API consumed by multiple different clients.

Q20 Server vs Client Components

MULTIPLE CHOICE

- ✅ Server Components run on the server and do not add JavaScript to the client bundle; Client Components run in the browser and can use React hooks and handle interactivity.

Q21 Server Components

SELECT ALL THAT APPLY

- ✅ They can fetch data directly using `fetch()` without needing `useEffect`.
- ✅ They reduce the amount of JavaScript sent to the browser.
- ✅ They can access server-side resources like databases and environment variables directly.

Q22 Registration Form

ESSAY

a) `react-hook-form` implementation

Use `useForm` with `zodResolver`, `register` to connect inputs, `handleSubmit` to wrap submission, and `formState.errors` for per-field messages:

```tsx
const { register, handleSubmit, formState: { errors } } = useForm<FormData>({
  resolver: zodResolver(schema),
});
// Display error:
{errors.email && <span>{errors.email.message}</span>}
```

b) Server Action — justification & data flow

Use a Server Action — the form is internal to the Next.js app; no external client needs a public URL.

1. User submits → client-side Zod validation runs.
2. If valid → Server Action called (`"use server"`).
3. Server re-validates, checks DB for duplicate email, hashes password, saves user.
4. Returns success → redirect, or returns error → displayed to user.

c) Why server-side validation is still required

> SECURITY Client-side validation can be bypassed entirely — e.g. via `curl` or DevTools. Without server-side validation, an attacker could POST `{ email: "x", password: "" }` directly to the Server Action endpoint, storing an empty password hash in the database.

Part VI · JavaScript Internals

Q23

Q23 The `this` Keyword

TRACE

LINE	THIS	WHY
`var gameOne = VideoGame('Zelda…', 1998)`	`window`	No `new` — defaults to the global object.
`var gameTwo = new Game('Fallout 1', 1997)`	`gameTwo`	`new` creates a new object and binds `this` to it.
`var gameThree = new VideoGame('Monkey Island…', 1991)`	`gameThree`	`new` creates a new object and binds `this` to it.
`gameTwo.play()`	`gameTwo`	Method call — `this` is the object before the dot.
`gameTwo.play.bind(empty).call()`	`empty`	`bind` permanently sets `this`; `.call()` cannot override it.
`setTimeout(gameThree.play, 1000)`	`window`	Detached callback — `this` loses its object context.

Appendix · Core Concepts

REFERENCE

A1 Redux Concepts

GLOSSARY

CONCEPT	DESCRIPTION
Connected Components	Components linked to the Redux store via `useSelector` / `useDispatch` (or `connect()`). They read state and dispatch actions without prop drilling.
Actions	Plain objects describing *what happened*: `{ type: 'cart/add', payload: item }`. The only way to signal a state change.
Reducers	Pure functions `(state, action) ⇒ newState` that define how state changes for each action type. No side effects allowed.
Dispatch	The method that sends an action to the store: `dispatch(action)`. Routes the action through reducers to produce new state.

A2 React Core Concepts

GLOSSARY

1 · WHAT IS A COMPONENT?

A self-contained, reusable function that accepts `props` and returns JSX describing UI.

2 · PRINCIPLES A COMPONENT SHOULD FOLLOW

- **Single Responsibility** — do one thing.
- **Pure / Predictable** — same props → same output.
- **Reusable & Composable** — generic, nestable.
- **Encapsulated** — clean props interface, internal state hidden.

3 · BENEFITS OF COMPONENTS

- **Reusability** — write once, use anywhere.
- **Maintainability** — isolated changes.
- **Testability** — easy to unit test in isolation.

A3 `background-size` Reference

GLOSSARY

VALUE	RESULT
`auto`	Image shown at its **natural / original size**.
`cover`	Scaled to **fill the container**, cropping if needed, aspect ratio preserved.
`contain`	Scaled to **fit entirely inside** the container, aspect ratio preserved, may leave gaps.
`100% 100%`	**Stretched** to fill exactly — aspect ratio distorted.
`50px 50px`	Displayed at the **exact pixel size** specified.